

Vel Tech Group of Institutions

Name: Gontu Yaswanth

Email: vtu33434@veltech.edu.in

Roll no: Vtu33434

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Competitive Coding II_L8

VTU_Week 3_COD

Attempt : 1

Total Mark : 50

Marks Obtained : 46.5

Section 1 : Coding

1. Problem Statement

Anil is working on a project that involves counting occurrences of a specific pattern within a given text. He needs a program that efficiently finds and counts all occurrences of a pattern within a given text using the Knuth-Morris-Pratt (KMP) algorithm.

Your task is to help Anil by developing a program that takes a text and a pattern as input and counts the number of times the pattern appears in the text using the KMP algorithm.

Input Format

The first line contains a string txt, representing the text in which Anil wants to search for the pattern.

The second line contains a string pat, representing the pattern Anil wants to search for in the text.

Output Format

The output displays a single integer, representing the count of occurrences of the pattern pat in the text txt.

If the pattern is not found, nothing will be printed.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: AABAACAADAABAABA
AABA

Output: 3

Answer

// You are using GCC

#include <stdio.h>

#include <string.h>

// Function to compute LPS array

```
void computeLPS(char pat[], int M, int lps[]) {  
    int len = 0;  
    lps[0] = 0;
```

```
    int i = 1;  
    while (i < M) {  
        if (pat[i] == pat[len]) {  
            len++;  
            lps[i] = len;  
            i++;  
        } else {  
            if (len != 0) {  
                len = lps[len - 1];  
            } else {  
                lps[i] = 0;  
                i++;  
            }  
        }  
    }  
}
```

```
}  
}  
}  
  
// Function to count occurrences using KMP
```

```
int KMPSearch(char txt[], char pat[]) {
```

```
    int M = strlen(pat);
```

```
    int N = strlen(txt);
```

```
    int lps[M];
```

```
    computeLPS(pat, M, lps);
```

```
    int i = 0, j = 0;
```

```
    int count = 0;
```

```
    while (i < N) {
```

```
        if (pat[j] == txt[i]) {
```

```
            i++;
```

```
            j++;
```

```
        }
```

```
        if (j == M) {
```

```
            count++;
```

```
            j = lps[j - 1];
```

```
        } else if (i < N && pat[j] != txt[i]) {
```

```
            if (j != 0)
```

```
                j = lps[j - 1];
```

```
            else
```

```
                i++;
```

```
        }
```

```
    }
```

```
    return count;
```

```
}
```

```
int main() {
```

```
    char txt[51], pat[26];
```

```
    scanf("%s", txt);
```

```
    scanf("%s", pat);
```

```
    int result = KMPSearch(txt, pat);
```

```
    if (result > 0)
        printf("%d", result);

    return 0;
}
```

Status : Correct

Marks : 10/10

2. Problem Statement

Rohan is developing a text analysis tool to efficiently find repeated patterns inside large documents. To speed up the searching process, he uses the Boyer–Moore string matching algorithm, specifically relying on the Bad Character Heuristic as the main data structure/technique.

Given a text string and a pattern string, write a program to ensure that the task is done by counting how many times the pattern occurs in the text.

Input Format

The first line contains a string representing the text (may contain spaces and special characters).

The second line contains a string representing the pattern to be searched.

Output Format

The output prints a single integer representing the number of occurrences of the pattern in the text.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: ababababa

ab

Output: 4

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#define MAX 256
```

```
// Bad Character Heuristic
```

```
void badCharHeuristic(char *pat, int m, int badchar[]) {
```

```
    for (int i = 0; i < MAX; i++)
```

```
        badchar[i] = -1;
```

```
    for (int i = 0; i < m; i++)
```

```
        badchar[(int)pat[i]] = i;
```

```
}
```

```
// Boyer-Moore Search
```

```
int boyerMoore(char *txt, char *pat) {
```

```
    int n = strlen(txt);
```

```
    int m = strlen(pat);
```

```
    int badchar[MAX];
```

```
    badCharHeuristic(pat, m, badchar);
```

```
    int count = 0;
```

```
    int s = 0; // shift
```

```
    while (s <= (n - m)) {
```

```
        int j = m - 1;
```

```
        while (j >= 0 && pat[j] == txt[s + j])
```

```
            j--;
```

```
        if (j < 0) {
```

```
            count++;
```

```
            // Shift for overlapping also
```

```
            s += (s + m < n) ? m - badchar[txt[s + m]] : 1;
```

```
        } else {
```

```
            int shift = j - badchar[txt[s + j]]; 
```

```
            s += (shift > 1) ? shift : 1;
```

```
        }
```

```
    }
```

```

    return count;
}

int main() {
    char txt[101], pat[101];

    // Read full line including spaces
    scanf("%[^\n]", txt);
    scanf("%[^\n]", pat);

    int result = boyerMoore(txt, pat);

    printf("%d", result);

    return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

Kiran is working on a string analysis module where he needs to identify repeating patterns efficiently. To achieve this, he uses the LPS (Longest Prefix Suffix) array from the KMP string matching algorithm as the main data structure.

Given a string, write a program to ensure that the task is done by finding the length of the longest proper prefix that is also a suffix, ensuring the prefix and suffix do not overlap.

Example

Input:

aabcdaabc

Output:

4

Explanation:

The string "aabc" is the longest prefix which is also a suffix. So the length of the string "aabc" is printed as 4.

Input Format

The input consists of a string S, representing the input string.

Output Format

The output prints an integer representing the length of the longest proper prefix which is also a suffix.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: aabcd aabc

Output: 4

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main() {
```

```
    char s[101];
```

```
    int lps[101];
```

```
    scanf("%s", s);
```

```
    int n = strlen(s);
```

```
    lps[0] = 0;
```

```
    int len = 0, i = 1;
```

```
    while (i < n) {
```

```
        if (s[i] == s[len]) {
```

```
            lps[i++] = ++len;
```

```
        } else {
```

```

    if (len != 0)
        len = lps[len - 1];
    else
        lps[i++] = 0;
}

printf("%d", lps[n - 1]);
return 0;
}

```

Status : Partially correct

Marks : 6.5/10

4. Problem Statement

Liam is a software engineer who loves analyzing strings for patterns. One day, while reviewing usernames in a system, he noticed that some usernames read the same backward and forward (palindromes). He wants to count all palindromic sequences of characters in a username efficiently.

To accomplish this, Liam decides to use Manacher's Algorithm, which finds all palindromic substrings in linear time.

Given a string representing the username, help Liam count all the possible palindromic substrings using Manacher's Algorithm.

Input Format

The input consists of a single line containing the string S.

The string consists of lowercase English letters only.

Output Format

The output prints a single integer, representing the total number of palindromic substrings present in the username.

Refer to the sample input and output for format specifications.

Sample Test Case

Input: abaa

Output: 6

Answer

// You are using GCC

#include <stdio.h>

#include <string.h>

```
int main() {
```

```
    char s[105], t[210];
```

```
    scanf("%s", s);
```

```
    int n = strlen(s);
```

```
    // Transform string: ^#a#b#a#$
```

```
    t[0] = '^';
```

```
    int j = 1;
```

```
    for (int i = 0; i < n; i++) {
```

```
        t[j++] = '#';
```

```
        t[j++] = s[i];
```

```
    }
```

```
    t[j++] = '#';
```

```
    t[j++] = '$';
```

```
    int P[210] = {0};
```

```
    int center = 0, right = 0;
```

```
    int count = 0;
```

```
    for (int i = 1; i < j - 1; i++) {
```

```
        int mirror = 2 * center - i;
```

```
        if (i < right)
```

```
            P[i] = (P[mirror] < (right - i)) ? P[mirror] : (right - i);
```

```
        while (t[i + (1 + P[i])] == t[i - (1 + P[i])])
```

```
            P[i]++;
```

```
        if (i + P[i] > right) {
```

```
            center = i;
```

```
            right = i + P[i];
```

```

    }
    count += (P[i] + 1) / 2;
}

printf("%d", count);
return 0;
}

```

Status : Correct

Marks : 10/10

5. Problem Statement

Aryan is working on a text processing project where he needs to analyze strings to find patterns efficiently. He decides to use the Z-algorithm, which for each position in a string, computes the length of the longest substring starting from that position which is also a prefix of the string.

Your task is to implement a program that computes the Z-array for a given string.

Input Format

The input consists of a single line containing the string S.

S may contain only uppercase and lowercase English letters.

Output Format

The output prints the Z-array of S as a sequence of integers separated by spaces.

The i-th element of the Z-array represents the length of the longest substring starting at position i which matches the prefix of S.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: aabxaab

Output: 0 1 0 0 3 1 0

Answer

// You are using GCC

#include <stdio.h>

#include <string.h>

int main() {

char s[101];

int Z[101] = {0};

scanf("%s", s);

int n = strlen(s);

int L = 0, R = 0;

for (int i = 1; i < n; i++) {

if (i <= R)

Z[i] = (R - i + 1 < Z[i - L]) ? (R - i + 1) : Z[i - L];

while (i + Z[i] < n && s[Z[i]] == s[i + Z[i]])

Z[i]++;

if (i + Z[i] - 1 > R) {

L = i;

R = i + Z[i] - 1;

}

for (int i = 0; i < n; i++)

printf("%d ", Z[i]);

return 0;

}

Status : Correct

Marks : 10/10